Computing and Digital Skills Curriculum

Python, Git, SQL, REST APIs, FastAPI, and debugging reference material.

Reference material for lesson planning and course-content development.

Contents

1. Programming foundations and classroom expectations

This section sets the baseline for introductory computing lessons. Learners should be comfortable reading short Python programs before they are asked to build larger ones. The focus is on understanding what the program is doing, what data changes as it runs, and why a particular result appears. Examples should stay small enough to trace by hand. A learner who can explain a five-line program clearly is in a better position to write a twenty-line program than a learner who has only copied longer code.

Variables should be introduced as names that refer to values used by the program. Good early examples include a student's score, the price of an item, a username, or a running total. Avoid turning the first lesson into a tour of Python's whole type system. Strings, integers, floats, and booleans are enough for most beginner work. Learners should see that the value associated with a variable can change, and that the new value is what later lines use.

What learners should be able to do

- Read a short program from top to bottom and describe the value of important variables after each line.
- Distinguish assignment from comparison when both are later introduced.
- Predict the output of a small program before running it.
- Change one input or variable and explain why the result changes.

A useful teaching habit is to ask learners for a prediction before execution. This makes the computer's output evidence rather than magic. When a prediction is wrong, the difference gives the teacher something concrete to discuss. Debugging should be presented as a normal part of programming, not as a sign that the learner has failed.

2. Python loops

Loops are introduced after learners have seen variables, simple expressions, and if-statements. The main purpose is to replace repeated code with a rule for repetition. Start by showing a small piece of copied code such as three nearly identical print statements. Ask what is repeated and what changes. Then replace that repetition with a loop. The lesson should make the benefit visible before naming the concept.

For-loops

A for-loop is a good fit when the program is iterating over a known collection or a known sequence of values. Use range() carefully because beginners often assume the stopping value is included. Examples such as range(5), range(1, 6), and range(2, 11, 2) are useful because they show the start, stop, and step behavior without extra complexity.

- for i in range(5) produces 0, 1, 2, 3, 4.
- for i in range(1, 6) produces 1 through 5.
- for name in names is usually clearer than looping over indexes when the index itself is not needed.

While-loops

A while-loop is useful when repetition depends on a condition rather than a fixed number of items. The key classroom question is: what changes inside the loop so the condition can eventually become false? If the learner cannot answer that, the program may contain an infinite loop. Simple examples include retrying input until it is valid, counting down, or repeating until a balance reaches a target.

Counters and accumulators

A counter records how many times something happens. An accumulator builds a running total. These two patterns are worth separating because learners often overwrite a total instead of adding to it. A trace table with columns for loop value and total makes the change visible.

Step	Current value	Running total
Start	-	0
1	5	5
2	8	13
3	3	16

Common mistakes

- Using range(1, 5) when the requirement is to include 5.
- Forgetting to change the variable used in a while condition.
- Resetting an accumulator inside the loop instead of before the loop.
- Using break and continue as if they mean the same thing.
- Adding a nested loop before understanding what the inner loop does for each outer-loop iteration.

Practice should include at least one predict-the-output task, one debugging task, and one small program the learner writes. A good debugging example is short enough that the learner can point to the exact line causing the problem.

3. Python functions

Functions should be taught as a way to name and reuse a piece of logic. Begin with a repeated operation that learners already understand. If a program converts several Celsius temperatures to Fahrenheit, the repeated conversion is a natural candidate for a function. This gives the function a purpose before syntax is introduced.

Definition and call

Learners should distinguish defining a function from calling it. Defining tells Python what the function does. Calling asks Python to run it. This sounds simple, but beginners often expect a function to run because they can see its definition in the file.

Parameters and arguments

A parameter is the name used inside the function definition. An argument is the value supplied when the function is called. Keep examples concrete. In greet(name), name is the parameter. In greet('Amina'), 'Amina' is the argument.

Return versus print

One of the most important beginner distinctions is return versus print. print() displays something. return sends a value back to the caller. A function that only prints a calculated value is harder to reuse when another part of the program needs that value. Use examples where the caller stores the returned result in a variable.

- Ask learners what a function returns, not only what appears on screen.
- Include a broken example that prints where a return value is required.
- Keep scope discussion basic: variables created inside a function are not automatically available everywhere else.

Practice progression

A useful sequence is: identify the purpose of a function, predict a return value, fix a simple function, then write one. Avoid jumping directly to recursion, decorators, lambdas, or advanced argument patterns in an introductory lesson.

4. Python lists and iteration

Lists are a natural next step after variables because they let one name refer to an ordered collection of values. Start with examples learners already understand, such as a list of temperatures, student names, product prices, or task statuses.

Core operations

- Create a list with a few values.
- Read an item by index.
- Change an item by index.
- Append a new item.
- Remove an item by value or position.
- Loop directly over the values in a list.

The zero-based index should be demonstrated visually. A short table showing index 0 under the first value prevents repeated confusion later. Negative indexes may be mentioned only after the main indexing idea is secure.

Common mistakes

Learners often use an index that does not exist, confuse an element with its index, or modify a list while iterating over it without understanding the result. Keep modification-during-iteration examples out of the earliest exercises unless the goal is specifically to discuss why it can be risky.

Practice

Good list practice combines selection and iteration. Examples include finding values above a threshold, calculating an average, counting matching items, or building a new list from filtered values. The lesson should not become an algorithms class. The aim is to make list state and loop behavior visible.

5. Debugging and error reading

Debugging is not a separate skill that appears after programming. It should be part of every topic. Learners should be shown how to read the last line of a traceback, locate the referenced line, and describe what the program expected versus what it received.

Three useful error categories

- Syntax errors: Python cannot parse the program.
- Runtime errors: the program starts but fails while running.
- Logic errors: the program runs but produces the wrong result.

Logic errors deserve special attention because there may be no red traceback. A wrong JOIN condition, an accumulator reset in the wrong place, or an off-by-one range can all produce believable output. Learners should compare expected behavior with actual behavior.

Debugging routine

- Reproduce the problem with the smallest useful input.
- Read the error message or inspect the incorrect output.
- Check the variables or data involved at the point of failure.
- Change one thing at a time.
- Run the same case again and then try a second case.

Print statements are acceptable for beginner debugging when used deliberately. Later, logging and tests can replace many ad-hoc prints. The lesson should focus on observation and reasoning rather than memorizing specific debugging tools.

6. Git branches, commits, and pull requests

Git should be introduced as a record of changes and a way to work safely with others, not just as a remote backup. The learner should be able to explain what changed between two commits and why a branch exists.

Branches

A branch gives a developer a line of work that can change without immediately changing the main branch. Feature branches should have names that describe the work. The exact naming convention matters less than using it consistently.

Commits

A commit should represent one understandable change. Messages such as 'fix', 'update', or 'stuff' give almost no information later. Better messages describe the change: 'add API timeout handling' or 'fix loop range to include final day'.

Pull requests

A pull request explains what a branch changes and gives another person a review point. A useful description says what changed, how to test it, and what is not finished. This can be brief for small student work, but it should still be meaningful.

Merge conflicts

A merge conflict means Git cannot safely combine overlapping edits. Learners should read both sides, decide what the final file should contain, remove the conflict markers, test the result, then finish the merge. The objective is not to memorize a command sequence. It is to understand that a person must make a content decision.

Secrets

API keys belong in environment variables or an ignored .env file. Adding .env to .gitignore after committing it does not remove the key from earlier history. If a key has been committed, rotate it. This is worth practicing because deleting the latest copy can give a false sense that the secret is gone.

7. SQL SELECT, WHERE, JOIN, and GROUP BY

SQL lessons should start with a small schema learners can understand. A users table and an orders table are enough to teach several useful ideas. The objective is to see how rows are selected and connected, not to memorize every SQL feature.

	orders	
me, city	id, user_id, total, created_at	
Lahore	101, 1, 120, 2026-07-01	
a, Karachi	102, 1, 80, 2026-07-03	
		103, 2, 200, 2026-07-05

SELECT and WHERE

SELECT chooses what to return. WHERE filters rows. Learners should run queries directly and inspect the rows, then change one condition and predict the difference.

JOIN

A JOIN combines related rows using a condition. In the schema above, users.id = orders.user_id is the meaningful relationship. A query can still run if the wrong columns are joined. This is why a syntactically valid query is not automatically correct.

GROUP BY

GROUP BY collects matching rows into groups so aggregate functions can be used. COUNT can count orders per user. SUM can total order value per user. The grouping column should match the question being asked.

- Ask learners to predict how many rows a JOIN will produce.
- Include one wrong JOIN that looks plausible.
- Include one grouped query and ask what each result row represents.

Subqueries can be shown as an extension, but the core lesson should stay on SELECT, WHERE, JOIN, GROUP BY, and aggregates.

8. REST API basics

A REST API lesson should make the request and response visible. Learners often understand an endpoint much faster when they see the method, path, JSON body, status code, and response JSON together.

Requests

- GET usually retrieves data.
- POST usually creates or submits data.
- PATCH usually changes part of an existing resource.
- DELETE usually removes a resource.

These are conventions, not magic rules enforced by the internet. The API designer still chooses the behavior. The important point is that the method and path form a clear contract for the caller.

Responses

A status code tells the client what happened. 200 is normal success, 201 is often used after creation, 404 means a requested resource was not found, and 422 commonly appears in FastAPI when request validation fails. A 500-level response indicates a server-side problem.

Request and response bodies

JSON bodies should have predictable fields. A client should not have to guess whether the identifier will be named id, user_id, or customer every time. Consistency matters more than clever naming.

Method	Path	Typical purpose
GET	/lessons/42	Read lesson 42
POST	/lessons	Create a lesson
PATCH	/lessons/42	Update part of lesson 42
DELETE	/lessons/42	Delete lesson 42

9. FastAPI and Pydantic

FastAPI lets a Python function handle an HTTP route. Pydantic models describe the expected shape of request and response data. This makes validation visible and gives learners a practical reason to define data structures clearly.

Request validation

If a request model requires session_id and message, a request missing message should be rejected before the chat logic runs. This is easier to debug than allowing missing data to travel deeper into the application.

Useful beginner endpoints

- GET /health to confirm the server is running.
- POST /chat for a conversational request.
- GET /lessons/{id} or GET /notes/{id} for a saved resource.
- POST /lessons or POST /notes for a project-specific action.

Learners should test endpoints with Swagger, curl, or Postman. Reading the code is not enough. Send valid input, missing fields, wrong types, unknown IDs, and multiple session IDs.

Errors

A missing saved lesson should return a controlled not-found response rather than a generic server error. Provider timeouts and invalid API keys should also be caught so the caller receives a useful message.

10. Putting computing topics into lesson packs

When generating a lesson pack from computing reference material, the learning goal should stay narrower than the entire source library. A lesson on Python loops may retrieve a debugging section, a lesson-design section, and the specific loop curriculum, but it should not suddenly teach SQL or FastAPI because those documents are present.

Expected lesson-pack parts

- A clear title and audience level.
- Learning objectives that match the source outline.
- Short explanation sections with worked examples.
- Practice that moves from guided to independent.
- A quiz or knowledge check with answers and short explanations.
- A source list showing which reference material informed the lesson.

Beginner and advanced variants should preserve the main learning objective. The advanced version may reduce hints, combine previously taught ideas, or use a less familiar scenario. It should not change the topic into something unrelated.

For follow-up edits, keep the request local when possible. If the user asks to make the second example easier, the rest of the lesson should remain stable unless the requested change creates an inconsistency.